

Мультиагентная платформа цифровых корпоративных сотрудников

Защищённая платформа анализа корпоративных знаний с графовыми подходами

Александр Зубик · AI Architect · ООО «АртКлауд»

Выпускной проект OTUS AI Architect · сентябрь 2026

Что защищаю

ZuBrIQ

Продукт ▾ Ресурсы ▾ Модели ▾ Цены Организациям

☰

Чат

• GPT • CLAUDE • GEMINI • GROK • ZUBR

GPT, Claude, Gemini, Grok

и свои модели Zubr

В каталоге шлюза больше ста моделей: GPT, Claude, Gemini, Grok. Свои модели: Zubr, Zubr Coder и Zubr Vision. Договор с белорусским юридическим лицом (ООО «АртКлауд»), оплата в BYN. Без VPN и зарубежных карт.

Открыть чат бесплатно

Модели и цены

100+

моделей: GPT, Claude, Gemini, Grok

Zubr

собственные модели в закрытом контуре

BYN

договор и оплата в белорусских рублях

и ещё сотня

GPT-6 Astra

Gemini 3.8 Flash

Claude Fable 5.1

Qwen3.8 Flash

Zubr

— доступ к моделям

Свои Zubr и шлюз к моделям из каталога

закрытый контур

шлюз

Один ключ на контур и на шлюз. В контуре запрос не выходит из периметра. В шлюзе: GPT, Claude, Gemini, Grok и ещё 100 моделей, в том числе для разработки.

Чат

2

Задача из ТЗ

Спроектировать и реализовать **MVP конвейера извлечения знаний из неструктурированных данных** в закрытом контуре.

Проблема	Ответ ТЗ
Галлюцинации и потеря контекста	GraphRAG — нейро-символический подход
Разграничение доступа	Security-by-Design на уровне чанков и узлов графа

Роль: Platform Engineer & AI Architect.

Предмет проекта — закрытый контур

Платформа **гибридная**: рядом с закрытым контуром есть коммерческий режим доступа к проприетарным моделям через одного агрегатора.

Предмет выпускного проекта — закрытый контур: свои модели на своём инференсе, нулевой egress.

Критерий ТЗ «не принято, если используются облачные API» выполняется **физически**: внешние модели отсекаются правами ключа, проверка стоит до обращения к вендору.

Цель и задачи

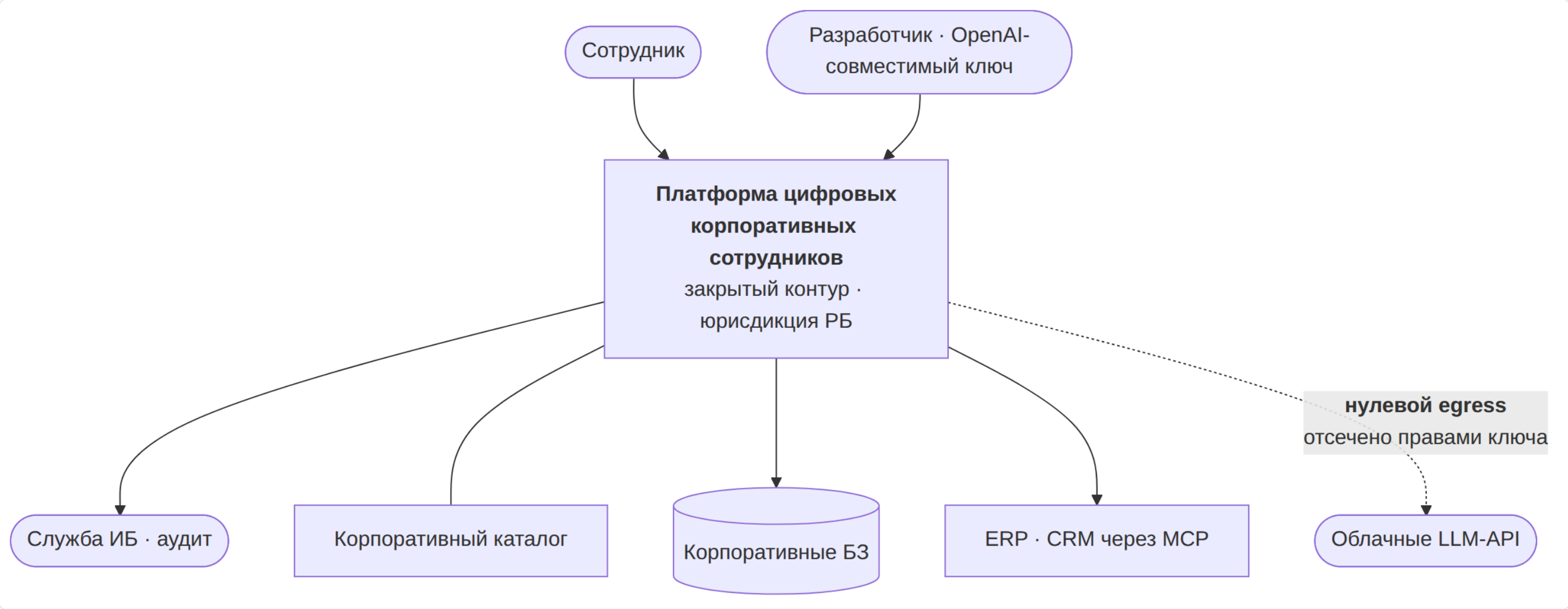
Цель: безопасный корпоративный ассистент, работающий полностью в контуре.

1. **GraphRAG** — граф связей документов, обход с учётом прав
2. **Security-by-Design** — права на уровне узлов, а не выдачи
3. **Мультиагентный слой** — роли-агенты со своими правами
4. **Наблюдаемость** — трейс по шагам, метрики пользы графа
5. **Доказать замерами**, а не декларациями

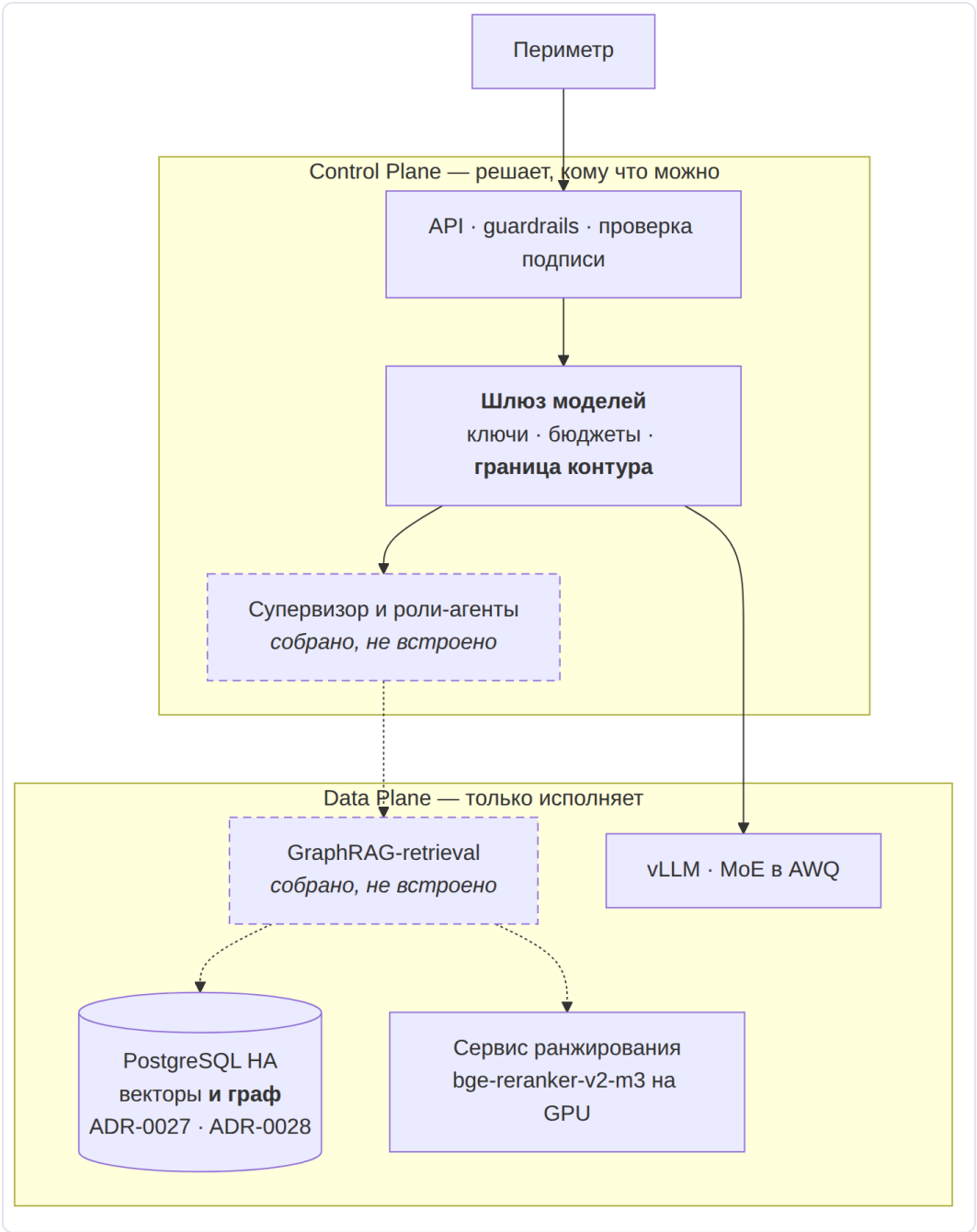
Критерии приёмки ТЗ

Критерий	Статус
Облачные API не используются	✓ предмет — закрытый контур, граница правами ключа
Диаграммы Deployment и Data Flow	✓ обе есть
Реализация GraphRAG (не просто вектор)	✓ + контрольный замер без графа
Security: «User B» не получает закрытый документ	✓ проверяется контекст модели, не только ответ
Control Plane / Data Plane	✓ в диаграммах и в коде
State machine, а не линейные скрипты	✓ цикл, ветвление, лимиты, checkpointer
Streaming · тесты на промпты	✓ SSE · golden set с гейтами

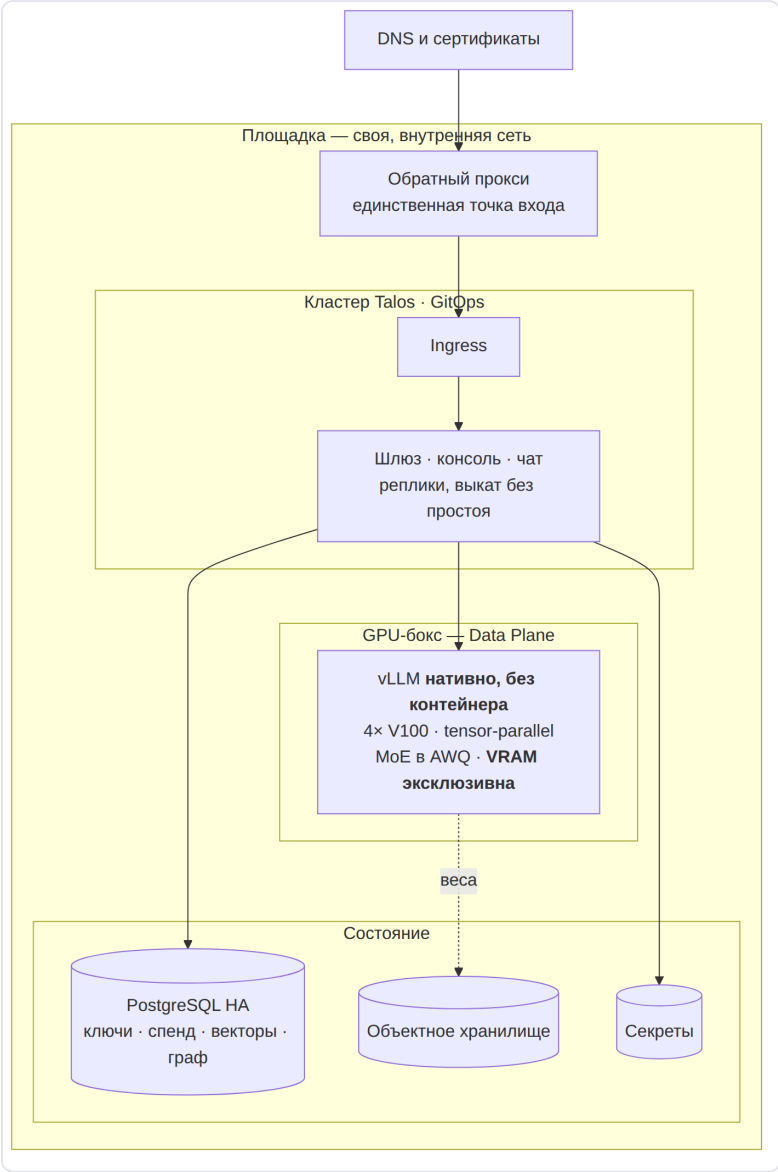
С4 · Контекст



С4 · Контейнеры

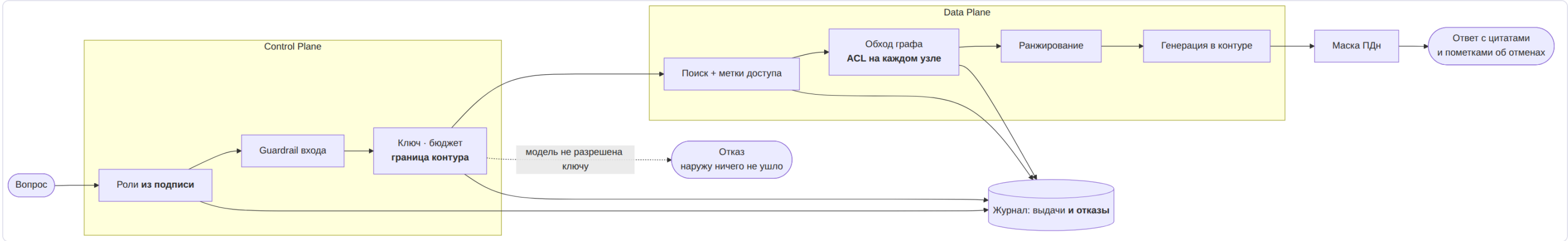


Deployment · где это физически живёт



Инференс в кластер не переносится. Карты в боксе, движок нативно — ради контроля над видеопамятью; кластер обращается к нему как к внешнему эндпоинту.

Data Flow · где поток решает про безопасность



Четыре решения о доступе — и все четыре **до** того, как данные куда-то уйдут.

Проблема, ради которой нужен граф

Вопрос: «Сколько дней основной ежегодный отпуск?»

- ЛПА-01 § 1: «**28** календарных дней» ← это находит вектор
- ЛПА-04 § 1: «с 1 апреля — **30** дней, пункт 1 ЛПА-01 отменяется»

Отменяющий документ текстуально похож на десятки других. Гарантии попадания в top-k нет.

Сотрудник получает отменённую норму. Без оговорки. С цитатой.

Ответ: граф связей ЛПА

```
( :Документ ) - [ :СОДЕРЖИТ ] -> ( :Чанк )  
( :Чанк ) - [ :ОТМЕНЯЕТ ] -> ( :Чанк )  
( :Чанк ) - [ :ССЫЛАЕТСЯ_НА ] -> ( :Документ )  
( :Чанк | :Документ ) - [ :ДОСТУПЕН_РОЛИ ] -> ( :Роль )
```

Retrieval: **вектор → обход 1–2 хопа → rerank**

Отмены попадают в контекст **безусловно**, вне конкуренции за места: модификатор может быть непохож на вопрос, но именно он делает ответ верным.

Доказательство: граф не декорация

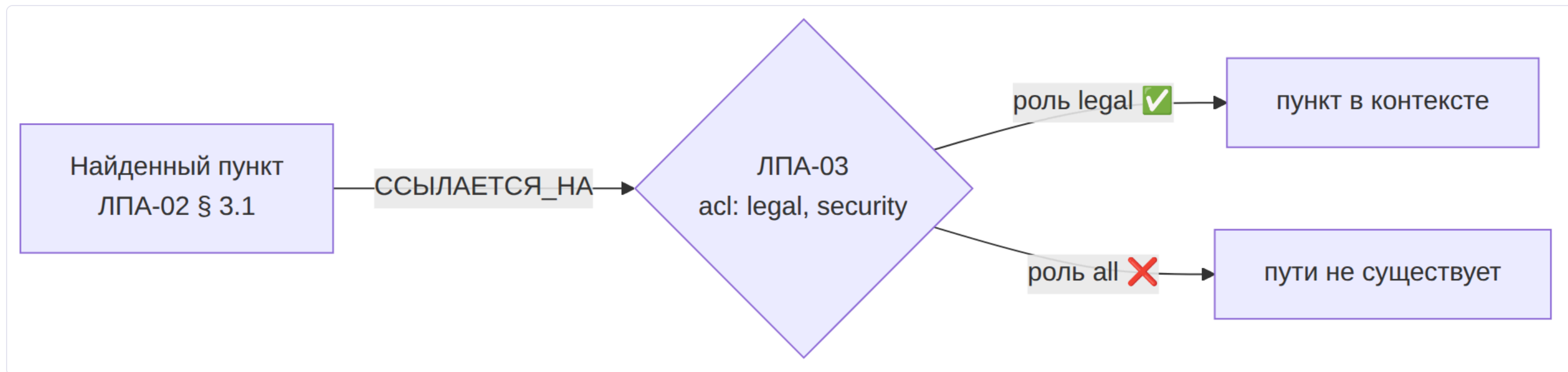
Контрольный замер. Тест `test_graph_disabled_loses_the_annotation`: выключаем граф — пометка об отмене исчезает.

На живой модели платформы:

Согласно действующей редакции, с 1 апреля 2026 года отпуск составляет **30 календарных дней**. Прежняя норма (28 дней) применению не подлежит.

Метрика в проде: доля ответов со сработавшим ребром `ОТМЕНЯЕТ`. Ноль = граф не работает, как бы хорошо он ни выглядел на демо.

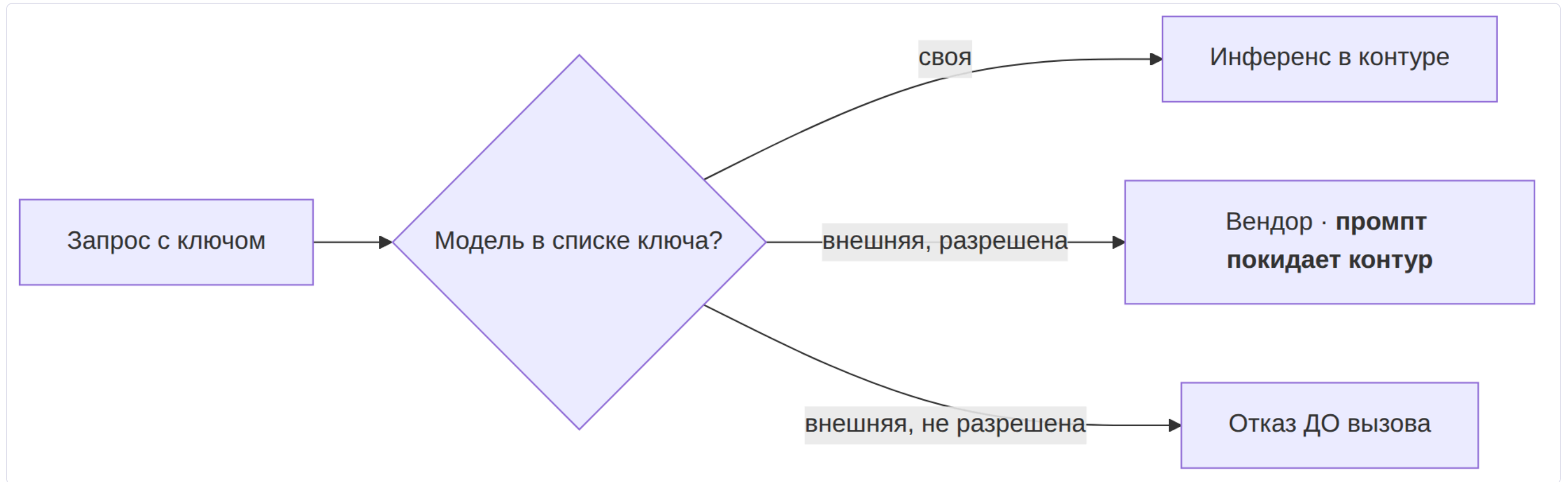
Security: ACL на каждом узле обхода



Предикат стоит **на каждом узле пути**, а не на финальной выдаче: иначе закрытый документ утекает через **структуру** — факт существования, название, соседей.

Следствие: **факт отмены не раскрывается**, если отменяющий документ недоступен субъекту.

Security: граница контура — одно поле у ключа



Барьер один, проверяем, и приложение не может его расширить.

Слабость названа честно: пустой список = видно всё. Умолчание работает в сторону разрешения — это первое, что надо перевернуть.

Security: роли берутся из подписи

Было: роли приходили **в теле запроса** — клиент сам объявлял, что ему можно.

Стало: только из подписанного токена; поле в теле на доступ не влияет вообще.

- фиксированный список асимметричных алгоритмов → подмена `RS256`→`HS256` и `alg: none` отсекаются
- проверка `exp`, `aud`, `iss`, кэш ключей с TTL
- 18 тестов; подделка собирается побайтово — библиотека её сделать не даёт

Мультиагентный слой — state machine



4 роли · маршрутизация **правилами** · лимиты шагов, инструментов и токенов · checkpointer по `request_id`

Права агента = права пользователя $\cap$ мандат роли, и пересечение считается в инструменте, а не в промпте.

MAS: честный результат замера

ADR-0015 требовал измерить три вещи перед принятием. Измерены все три:

Что	Результат
Точность маршрутизации	13 / 13
Прирост качества против монолита	нет (recall 1,0 = 1,0)
Стоимость	×3,7 латентности с генерацией

ADR обязывал: «если иерархия не даст прироста — честно зафиксировать в защите». **Фиксирую: не дала.**

Оправдан требованием ТЗ и **разделением прав**, но не качеством ответов.

Инфраструктура платформы



Потоковый ответ: источники раньше текста

```
event: meta      {"request_id": "a3f1..."}
event: sources   [{"doc_code": "ЛПА-01", "ordinal": "1"}, ...] ← сначала основания
event: token     "Основной"
event: token     " ежегодный"
event: note      "ЛПА-01 § 1 отменён документом ЛПА-04"
event: done
```

Источники уходят раньше первого слова. Для ассистента по нормативным документам цитата важнее скорости появления текста: пользователь сразу видит, на чём будет построен ответ, и может остановиться, если основание не то.

Регрессия, закрытая тестом: контекстные переменные **не переживают `yield`**. ASGI-сервер шагает генератор в разных контекстах, и обычная `with`-обёртка роняет спан. Родитель передаётся явно — трейс обязан остаться одним и закрытым.

Наблюдаемость: request_id открывает трейс

request_id — это uuid4().hex, ровно 128 бит trace_id OpenTelemetry. Он не прокидывается атрибутом, а **становится** идентификатором трейса.

ask	2368 мс	
retrieve.rerank	708 мс	{candidates: 14}
graph.expand + annot	5,5 мс	
llm.generate	1611 мс	{model: своя модель контура}

Содержимое запроса в трейс **не попадает**: трассировка доступна шире, чем сами документы.

Замеры: где на самом деле время

Шаг	Доля
Генерация (своя модель)	68 %
Реранк (cross-encoder на CPU)	30 %
Эмбеддинг	1,5 %
Обход графа	0,3 %

Без генерации реранк занимает **76 %**.

Вопрос «не тормозит ли ваш GraphRAG» закрыт цифрой: граф — три десятых процента.

Три ошибки вокруг одного числа

Пилот платформы: «нарезка с заголовком дала **+22 п.п. hit@1**». Я взял это как основание и разнёс по шести документам.

1. **В пилоте это не измеряли.** 67 % → 89 % — сравнение двух систем, различавшихся размером куска, нарезчиком и реализацией rerank. «Дело в нарезке» — догадка автора.
2. **Я распространил догадку как факт** — потому что она вела туда, куда я и так собирался.
3. **Обнаружив это, я снова не стал мерить.** Построил довод: «адресных вопросов 3 % — 22 пункта они не дадут». Логично, посылка верна, вывод неверен.

Скепсис без замера — такая же догадка, как утверждение. И изнутри ощущается так же убедительно.

Замер вместо рассуждения: 25 кодексов, 12 644 куска

Одна переменная — есть ли заголовок в теле куска. Протокол, нарезка и эталон платформы без изменений.

Тело куска	hit@1	hit@6
полный якорь (как в проде)	90,2 %	98,4 %
только название статьи	90,2 %	95,1 %
только адрес	75,4 %	96,7 %
без якоря	73,8 %	95,1 %

Якорь даёт +16,4 п.п. — догадка была верной. Но работает название, а не адрес: одно название даёт те же 90,2 %, один адрес — 75,4 %. Не в сумме: название адрес заменяет.

«Статья 164. Банковская гарантия» — метка, написанная человеком. Текст самой нормы этих слов рядом не содержит: норма описывает механизм, а не называет его.

Замер дал рекомендацию. Рекомендация оказалась не той

Отчёт заканчивался пунктом № 1: **«реранк — на ONNX и в отдельный пул»**. Логично: 76 % времени в одном компоненте.

Проверка разделяющей способности показала другое:

	Худший в корпусе	Лучший вне	Зазор
ms-marco-MiniLM	8,2658	8,9686	-0,70
bge-reranker-v2-m3	0,9919	0,0004	+0,9915

ONNX ускорил бы модель и **оставил бы главное**: её оценки не калиброваны, порог отказа на них невозможен.

А у платформы ранжирование **уже вынесено на GPU**. Держать свою модель там, где есть сервис, — не независимость, а дублирование.

Реранк отдан сервису платформы — ADR-0027

Один стенд, одна переменная окружения. healthz 452 против 456 RPS — контроль, что стенд не менялся.

Сценарий	Своя модель	Сервис платформы
ask, 1 клиент	1,32 RPS · 742 мс	5,32 RPS · 177 мс
ask, 8 клиентов	1,79 RPS · 4 369 мс	11,03 RPS · 719 мс
Доля реранка	96 %	68 %
Токенов контекста	739	604

Главное — не ×4, а изменившийся характер насыщения. Было: пропускная способность не растёт с параллелизмом (+36 % на восьмикратном росте клиентов) — torch занимает все ядра внутри вызова. Стало: дорогой шаг — ожидание сети, **+107 %**.

ADR-0004 «Vector DB — Qdrant» → superseded . Предметом решения было **не хранилище, а конвейер ранжирования** — ровно то, что за месяц до этого показал пилот платформы: «решает реранкер, а не хранилище».

Качество: golden set и гейты

14 кейсов, метрики считаются **без LLM** — объективно и воспроизводимо.

Метрика	Значение
Context recall	1,0
Нарушений ACL	0
Пометка об отмене	1,0
Честный отказ вне корпуса	1,0
Точность маршрутизации	13/13
Faithfulness (судья на другой модели)	0,923

Прогон возвращает код 1 при провале — готовый шаг CI. **Локальная конфигурация его возвращает:** после ужесточения гейта «честный отказ» она приёмку не проходит.

Что нашли замеры, а не тесты

Найдено	Было бы в проде
Обход в графе не набирал глубину	цепочка отмен обрывалась на первом звене
В запросе не было одного направления ребра	от отменяющего пункта не виден отменённый
Система отвечала на любой вопрос	ответ про спутники со ссылками на отпуск
Текст отказа выдавал существование закрытого документа	утечка через факт существования
Тест портил права в живой базе	демонстрация отмены ломалась молча
Сам эталон засчитывал шум как успех	«не протёк закрытый документ» ≠ «ответил правильно»

Последнее — про кейс, где ответа для роли в корпусе нет. Эталон проверял только утечку и принимал выдачу из **шести посторонних пунктов**. Проверено по корпусу: журнал аудита упомянут ровно один раз, и в закрытом документе. `refusal_pass` стал гейтом.

Граф: Neo4j или та же СУБД, где уже лежат векторы

Три бэкенда за одним интерфейсом, один корпус, четыре критерия. Три из четырёх не разделили.

Критерий	Neo4j	PostgreSQL
Качество выдачи	0 расхождений	0 расхождений
Обход при hops=2	1,5 мс	2,4 мс
Обход при hops=11	13,2 мс	2,6 мс
Изоляция окружений	дисциплина	конвенция

Решает онтология и предикат доступа, а не движок. Тот же вывод, что пилот платформы сделал про векторы: «решает реранкер, а не хранилище».

Neo4j Community: база одна на инстанс — CREATE DATABASE не поддерживается. Тест, стейдж и прод делят пространство имён.

Сравнение нашло дефект в том, что уже работало

Первый прогон: у Neo4j **12 расхождений** — при зелёных собственных тестах.

ЛПА-03 § 4 → ЛПА-03 § 1 ребро от ПРЕЖНЕЙ редакции правил

`build` работал на одних `MERGE` — это **доливка, а не пересборка**. Что перестало следовать из корпуса, живёт вечно и молча добавляет пункт в обход.

Снос пришлось ограничить кодами документов: инстанс общий, `DETACH DELETE` вынес бы чужое. У Postgres развилки нет — своя база, `TRUNCATE` безопасен.

Ограничение «база одна на инстанс» заставило написать менее правильный код. Это и есть аргумент изоляции — на живом дефекте, а не в рассуждении.

Зачем граф: спрос легко измерить не там

Платформа измерила спрос на исторические вопросы к праву: **0 из 3 738** сообщений пользователей. Вывод напрашивался — механизм есть, спроса нет.

Вывод неверен: смешаны два класса вопросов.

	«Что действовало до поправки?»	«Сколько дней отпуска?»
спрос	0 из 3 738	каждый вопрос к изменённой норме
без графа	нет ответа	уверенно неверный ответ

граф ВКЛЮЧЁН ЛПА-01 § 1 · пометка: отменён документом ЛПА-04

граф ВКЛЮЧЁН ЛПА-01 § 1 · пометка: нет «29 дней» вместо 30

Граф нужен **не** затем, чтобы отвечать на исторические вопросы, а затем, чтобы **не выдавать отменённую норму за действующую**.

И сразу оговорка, без которой это становится обещанием: гарантия покрывает только отмены, **известные корпусу**. Акт, отменённый тем, чего не загружали, останется действующим — граф этого не заметит по построению.

А на боевом корпусе платформы механизм сверки с источником **существует и написан правильно** — и сверялись им **10 %** документов, а 29 % не сверятся никогда: у них нет адреса источника. По коду этого не видно.

Экономика: решает утилизация

ТСО закрытого контура ≈ **\$42,5k за три года** на фактическом железе.

Утилизация	Цена 1М выходных токенов
100 %	\$0,89
25 %	\$3,56
5 %	\$17,78

Разброс двадцатикратный. Своё дешевле покупного только выше ~25 %.

Ниже — платим за простой. И покупаем при этом не цену, а юрисдикцию.

Чего сейчас нет

- **граф и агентный слой в платформе** — собраны и протестированы, **не встроены**
- **мультимодальный ingestion** — спроектирован, конвейера нет
- **эмбеддер остался на CPU** — после переноса реранка он второй по стоимости: 12 % ответа
- **сквозной трейсинг** через продукты
- **алерты с runbook'ами**, ретеншн следов
- **red teaming по методике занятия 33** — описан, не применялся
- **утилизация инференса не измерена** — главный неизвестный в экономике

Дорожная карта

1. **Встроить граф и агентный слой в платформу** — интерфейсы уже совместимы
2. Перевернуть умолчание прав ключа: пусто = только свои модели
3. Тест на схему БД шлюза в CI — иначе счета сломаются молча
4. ~~Реранк на ONNX~~ → **сделано иначе: сервисом платформы** ([ADR-0027](#)). Следующий — эмбеддер на `bge-m3` там же, с переиндексацией
5. Измерить утилизацию инференса — без неё экономика остаётся диапазоном
6. Ретеншн и режим доступа к трассировке
7. Red teaming с ASR-гейтом в CI

Выводы

- **Цель достигнута:** четыре блокирующих критерия T3 закрыты и подтверждены тестами
- **Работа делалась на реальной системе** — поэтому в ней есть и техдолг, и отрицательные результаты
- **Трудное:** не архитектура, а честные замеры. Каждый прогон находил то, чего не видели тесты
- **Полезность курса: 9 / 10** — методики уроков 13–24 использованы напрямую
- **Главный урок:** архитектура, не подтверждённая замером, — это гипотеза

75 тестов · 3 отчёта с цифрами · 26 ADR · один честно зафиксированный отрицательный результат

Спасибо

Вопросы?

Александр Зубик · AI Architect · ООО «АртКлауд»

Демонстрация: граф в браузере Neo4j · трейс по `request_id` · дашборд · потоковый ответ